

LAB 1

DevOps Foundations & Continuous Integration

Project 1 — Record of Commands & Execution Evidence

Student	Kunal
Environment	Ubuntu 24.04.4 LTS (WSL2)
Git	2.43.0
Java	OpenJDK 21.0.12
Jenkins	2.568.3
Project	lab1-devops
GitHub	lab1-devops-version-control

Purpose

This report consolidates the actual command-line, Git, Jenkins, pipeline, deployment and verification evidence produced during Lab 1. Existing evidence from the completed Lab 1 tasks is reused where it directly supports the Task 1 requirement.

1. Environment & Tool Verification

The Ubuntu environment and core DevOps tools were verified from the project terminal.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ cd ~/lab1-devops
echo "==== UBUNTU ===="
lsb_release -a

echo "==== USER ===="
whoami

echo "==== DATE ===="
date

echo "==== GIT ===="
git --version

echo "==== JAVA ===="
java -version

echo "==== JENKINS ===="
jenkins --version
==== UBUNTU ====
No LSB modules are available.
Distributor ID: Ubuntu
Description:   Ubuntu 24.04.4 LTS
Release:      24.04
Codename:     noble
==== USER ====
kunal_prajapati
==== DATE ====
Wed Sep  9 17:11:56 UTC 2026
==== GIT ====
git version 2.43.0
==== JAVA ====
openjdk version "21.0.12" 2026-07-21
OpenJDK Runtime Environment (build 21.0.12+8-1-24.04-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12+8-1-24.04-Ubuntu, mixed mode, sharing)
==== JENKINS ====
2.568.3
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ |
```

Figure 1 — Ubuntu 24.04.4 LTS, user identity, date, Git, Java and Jenkins versions.

Commands

```
cd ~/lab1-devops
lsb_release -a
whoami
date
git --version
java -version
jenkins --version
```

Observation: the required Linux environment and tool versions are available for the experiments.

2. Git Configuration & Java Evidence

Git global identity was configured for commits, and Java 21 was verified as the Jenkins runtime.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git --version
git version 2.43.0
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git config --global --list
user.name=Kunal
```

Figure 2 — Git 2.43.0 and global Git configuration.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ java -version
openjdk version "21.0.12" 2026-07-21
OpenJDK Runtime Environment (build 21.0.12+8-1-24.04-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12+8-1-24.04-Ubuntu, mixed mode, sharing)
```

Figure 3 — OpenJDK 21.0.12 verification.

Configuration

```
git config --global user.name "Kunal"
git config --global user.email "kunalprajapati14042005@gmail.com"
git config --global --list
```

3. Jenkins Installation & Service

Jenkins was verified as installed, enabled and actively running.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ jenkins --version
2.568.3
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ sudo systemctl status jenkins --no-pager
[sudo] password for kunal_prajapati:
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-09-09 12:46:50 UTC; 4h 19min ago
     Main PID: 2130 (java)
       Tasks: 65 (limit: 8276)
      Memory: 1002.5M (:)
     CGroup: /system.slice/jenkins.service
             └─2130 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkin...

Sep 09 12:54:34 LAPTOP-45509TK7 jenkins[2130]: 2026-09-09 12:54:34.940+0000 [id=289] INFO jenkins...plugins
Sep 09 12:54:34 LAPTOP-45509TK7 jenkins[2130]: 2026-09-09 12:54:34.943+0000 [id=277] INFO jenkins...plugins
Sep 09 12:54:34 LAPTOP-45509TK7 jenkins[2130]: 2026-09-09 12:54:34.975+0000 [id=286] INFO jenkins...plugins
Sep 09 12:54:34 LAPTOP-45509TK7 jenkins[2130]: 2026-09-09 12:54:34.979+0000 [id=284] INFO jenkins...ensions
Sep 09 12:54:34 LAPTOP-45509TK7 jenkins[2130]: 2026-09-09 12:54:34.995+0000 [id=277] INFO jenkins... loaded
Sep 09 12:54:35 LAPTOP-45509TK7 jenkins[2130]: 2026-09-09 12:54:35.007+0000 [id=277] INFO jenkins...adapted
Sep 09 12:54:35 LAPTOP-45509TK7 jenkins[2130]: 2026-09-09 12:54:35.021+0000 [id=288] INFO jenkins...ll jobs
Sep 09 12:54:35 LAPTOP-45509TK7 jenkins[2130]: 2026-09-09 12:54:35.022+0000 [id=278] INFO jenkins...updated
Sep 09 12:54:35 LAPTOP-45509TK7 jenkins[2130]: 2026-09-09 12:54:35.669+0000 [id=287] INFO jenkins...ization
Sep 09 12:54:35 LAPTOP-45509TK7 jenkins[2130]: 2026-09-09 12:54:35.671+0000 [id=110] INFO h.m.Upda... 55 sec
Hint: Some lines were ellipsized, use -l to show in full.
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ |
```

Figure 4 — Jenkins 2.568.3 version and active service evidence.

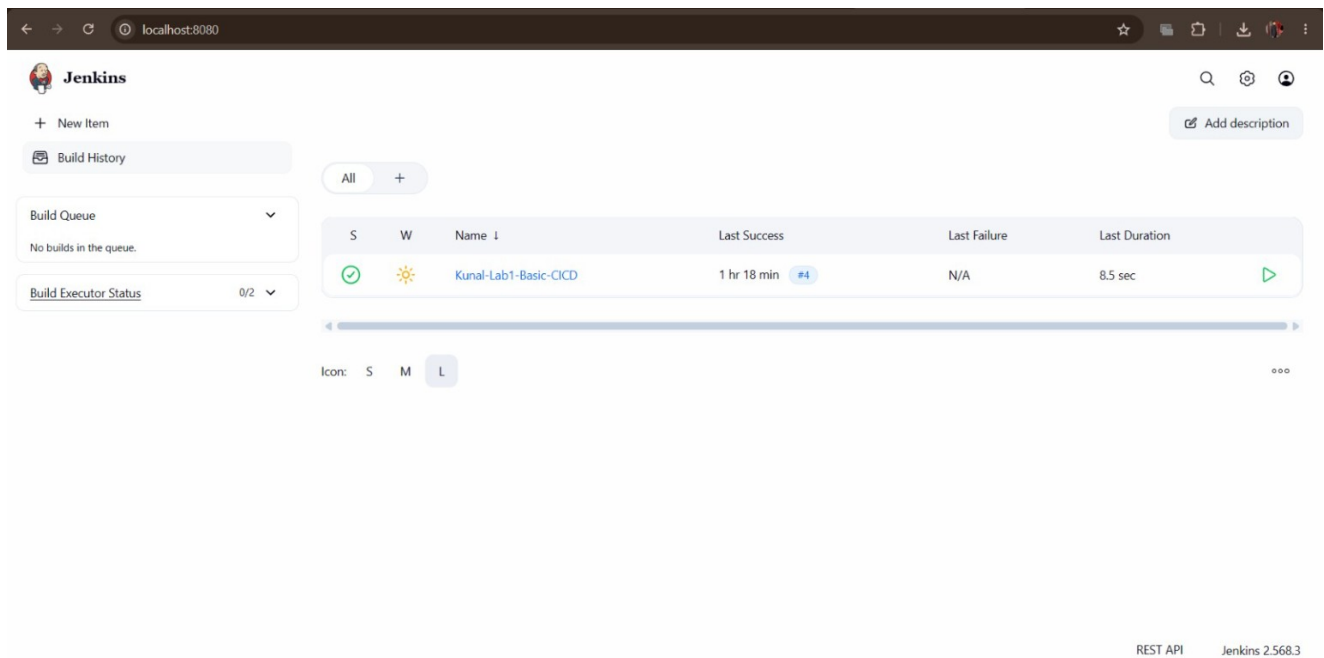
Command

```
jenkins --version
sudo systemctl status jenkins --no-pager
```

Observation: Jenkins service status is active (running) and enabled.

4. Jenkins Dashboard

The Jenkins web interface was accessible locally and displayed the Lab 1 job.



The screenshot shows the Jenkins dashboard at localhost:8080. The dashboard includes a sidebar with links to 'New Item', 'Build History', 'Build Queue', and 'Build Executor Status'. The main area displays a table of jobs. The table has columns for 'S', 'W', 'Name', 'Last Success', 'Last Failure', and 'Last Duration'. The job 'Kunal-Lab1-Basic-CICD' is listed with a green checkmark in the 'S' column, a sun icon in the 'W' column, and a duration of '8.5 sec'. The 'Last Success' column shows '1 hr 18 min' and the 'Last Failure' column shows 'N/A'. The dashboard also includes a 'Build Queue' section showing 'No builds in the queue' and a 'Build Executor Status' section showing '0/2'. The bottom right corner of the dashboard shows 'REST API' and 'Jenkins 2.568.3'.

S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	Kunal-Lab1-Basic-CICD	1 hr 18 min	N/A	8.5 sec

Figure 5 — Jenkins dashboard at localhost:8080 with Kunal-Lab1-Basic-CICD visible.

Observation: the Jenkins controller is operational and the configured job is available from the dashboard.

5. Project Workspace & Repository State

The Lab 1 work is maintained in ~/lab1-devops with Git metadata, the application directory, README and Jenkinsfile.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ cd ~/lab1-devops
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ pwd
/home/kunal_prajapati/lab1-devops
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ ls -la
total 28
drwxr-xr-x 5 kunal_prajapati kunal_prajapati 4096 Sep  9 17:04 .
drwxr-x--- 6 kunal_prajapati kunal_prajapati 4096 Sep  9 17:06 ..
drwxr-xr-x 8 kunal_prajapati kunal_prajapati 4096 Sep  9 15:40 .git
drwxr-xr-x 5 kunal_prajapati kunal_prajapati 4096 Sep  9 17:04 .venv
-rw-r--r-- 1 kunal_prajapati kunal_prajapati 1123 Sep  9 15:38 Jenkinsfile
-rw-r--r-- 1 kunal_prajapati kunal_prajapati 1186 Sep  9 15:36 README.md
drwxr-xr-x 2 kunal_prajapati kunal_prajapati 4096 Sep  9 15:36 app
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .venv/

nothing added to commit but untracked files present (use "git add" to track)
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ |
```

Figure 6 — Project workspace listing and Git status.

Commands

```
cd ~/lab1-devops
pwd
ls -la
git status
```

6. Git Commit History

The repository contains meaningful commits for setup, documentation, pipeline work and deployment.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git log --oneline --all --decorate --graph
* 8287c6b (HEAD -> main, origin/main) Automate application deployment with Jenkins
* f69e903 Add deployable web application
* 204d832 Enhance README with lab details and tools used
* 84e1e12 Add Jenkins CI/CD pipeline
* f7e9d4f (feature-notes) Add version control notes
* ee06e58 Initial project setup
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ |
```

Figure 7 — Git history on main and feature-notes.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git branch -M main
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git switch -c feature-notes
Switched to a new branch 'feature-notes'
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ echo "Git version control experiment completed." >> README.md
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git add README.md
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git commit -m "Add version control notes"
[feature-notes f7e9d4f] Add version control notes
1 file changed, 1 insertion(+)
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git log --oneline --all --decorate
f7e9d4f (HEAD -> feature-notes) Add version control notes
ee06e58 (main) Initial project setup
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ |
```

Figure 8 — Feature branch and commit history evidence.

Command

```
git log --oneline --all --decorate --graph
```

7. Git Merge & Repository Progress

The feature work was integrated and the repository history was retained as part of the version-control experiment.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git switch main
Switched to branch 'main'
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git merge feature-notes
Updating ee06e58..f7e9d4f
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git log --oneline --all --decorate --graph
* f7e9d4f (HEAD -> main, feature-notes) Add version control notes
* ee06e58 Initial project setup
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ |
```

Figure 9 — Git merge/history evidence.

Observation: the history preserves separate commits and demonstrates branch-based version control.

8. GitHub Remote & Push

The local project is linked to the GitHub repository and changes were pushed to main.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git remote -v
origin  https://github.com/kunalprajapati14042005/lab1-devops-version-control.git (fetch)
origin  https://github.com/kunalprajapati14042005/lab1-devops-version-control.git (push)
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .venv/

nothing added to commit but untracked files present (use "git add" to track)
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ |
```

Figure 10 — GitHub origin remote and repository status.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git push -u origin main
Username for 'https://github.com': kunalprajapati14042005
Password for 'https://kunalprajapati14042005@github.com':
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (6/6), 549 bytes | 549.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/kunalprajapati14042005/lab1-devops-version-control.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ |
```

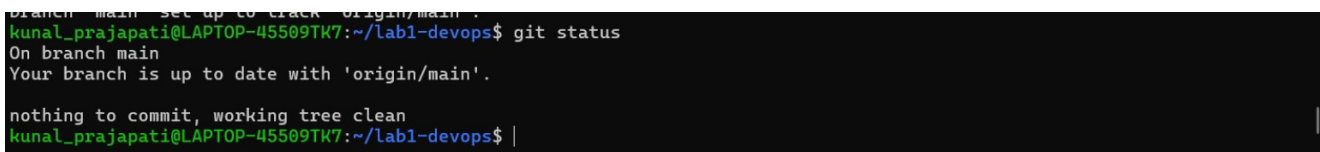
Figure 11 — Push operation to the GitHub main branch.

Commands

```
git remote -v
git push origin main
```

9. Final Git Synchronization

The final local repository state is synchronized with origin/main.

A terminal window with a black background and green text. The text shows the output of the 'git status' command. It indicates that the 'main' branch is set up to track 'origin/main', the user is on the 'main' branch, and the branch is up to date. It also states that there is nothing to commit and the working tree is clean.

```
branch 'main' set up to track 'origin/main'.
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ |
```

Figure 12 — Final Git status after synchronization.

Observation: the main branch is up to date with origin/main, providing a clean version-control checkpoint.

10. Jenkins Basic CI/CD Pipeline Configuration

The basic Jenkins job checks out the main branch from GitHub and executes an automated build stage.

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

```
5 stage('Checkout') {
6     steps {
7         git branch: 'main',
8           url: 'https://github.com/kunalprajapati14042005/lab1-devops-version-control.git'
9     }
10 }
11
12 stage('Build') {
13     steps {
14         sh 'echo "Automated build completed successfully."'
15         sh 'ls -la'
16     }
17 }
18 }
19 }
```

Save

Apply

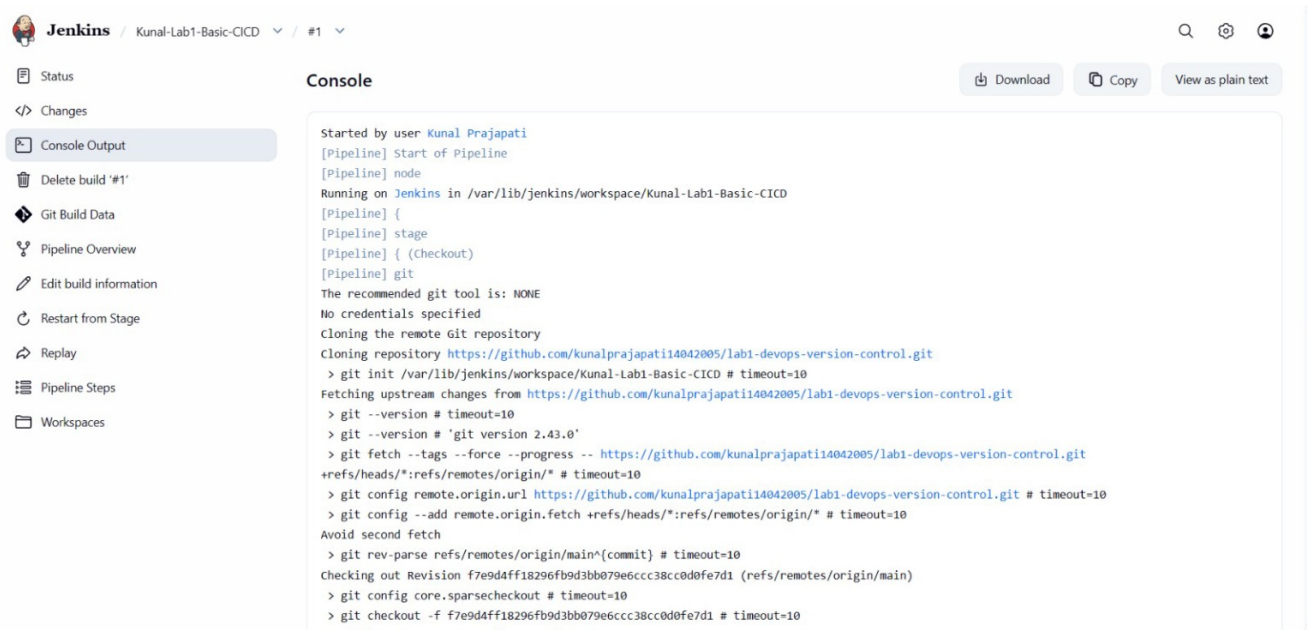
Figure 13 — Jenkins Pipeline configuration showing Checkout and Build stages.

Pipeline flow

GitHub main → Checkout → Build → SUCCESS

11. Jenkins Pipeline Execution

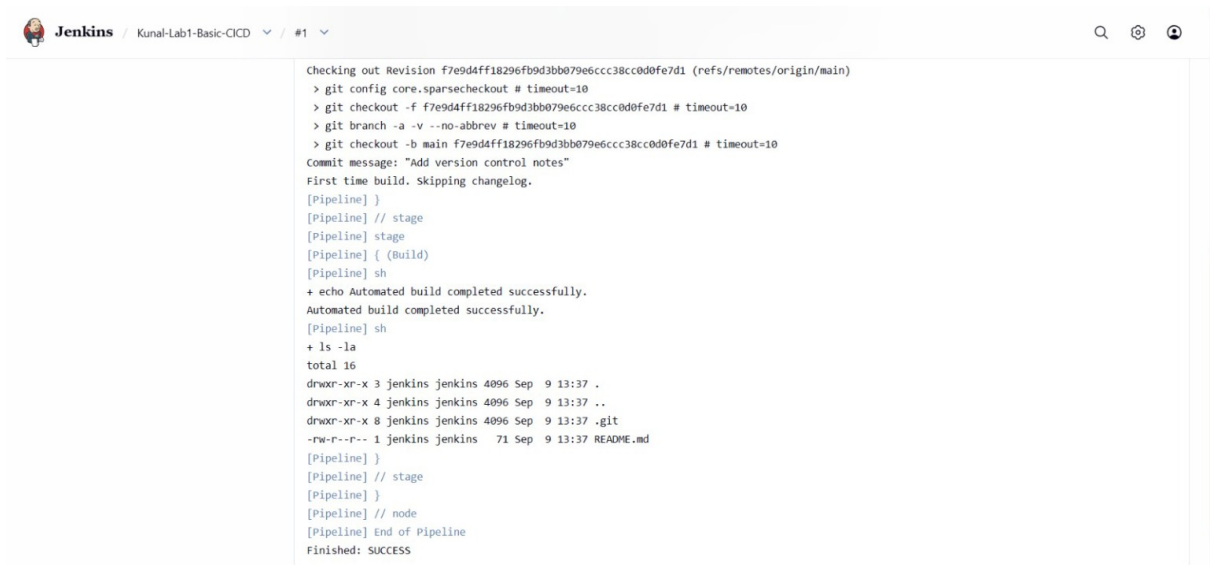
The Jenkins job successfully checked out the repository and completed the build stage.



The screenshot shows the Jenkins console output for a job named 'Kunal-Lab1-Basic-CICD'. The left sidebar contains navigation links: Status, Changes, Console Output (selected), Delete build '#1', Git Build Data, Pipeline Overview, Edit build information, Restart from Stage, Replay, Pipeline Steps, and Workspaces. The console output shows the following steps:

```
Started by user Kunal Prajapati
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Kunal-Lab1-Basic-CICD
[Pipeline] {
[Pipeline] stage
[Pipeline] { (checkout)
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/kunalprajapati14042005/lab1-devops-version-control.git
> git init /var/lib/jenkins/workspace/Kunal-Lab1-Basic-CICD # timeout=10
Fetching upstream changes from https://github.com/kunalprajapati14042005/lab1-devops-version-control.git
> git --version # timeout=10
> git --version # 'git version 2.43.0'
> git fetch --tags --force --progress -- https://github.com/kunalprajapati14042005/lab1-devops-version-control.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/kunalprajapati14042005/lab1-devops-version-control.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision f7e9d4ff18296fb9d3bb079e6ccc38cc0d0fe7d1 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f f7e9d4ff18296fb9d3bb079e6ccc38cc0d0fe7d1 # timeout=10
...
```

Figure 14 — Jenkins console output for checkout and pipeline execution.



The screenshot shows the continuation of the Jenkins console output, showing the build stage and completion:

```
Checking out Revision f7e9d4ff18296fb9d3bb079e6ccc38cc0d0fe7d1 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f f7e9d4ff18296fb9d3bb079e6ccc38cc0d0fe7d1 # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git checkout -b main f7e9d4ff18296fb9d3bb079e6ccc38cc0d0fe7d1 # timeout=10
Commit message: "Add version control notes"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] sh
+ echo Automated build completed successfully.
Automated build completed successfully.
[Pipeline] sh
+ ls -la
total 16
drwxr-xr-x 3 jenkins jenkins 4096 Sep  9 13:37 .
drwxr-xr-x 4 jenkins jenkins 4096 Sep  9 13:37 ..
drwxr-xr-x 8 jenkins jenkins 4096 Sep  9 13:37 .git
-rw-r--r-- 1 jenkins jenkins  71 Sep  9 13:37 README.md
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 15 — Successful Jenkins build evidence.

Observation: Jenkins completed the configured pipeline without errors.

12. Jenkinsfile from Source Control

The repository Jenkinsfile defines Build, Test and Deploy stages and post-build handling.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ cat Jenkinsfile
pipeline {
  agent any

  stages {
    stage('Build') {
      steps {
        echo 'Build stage completed successfully.'
      }
    }

    stage('Test') {
      steps {
        echo 'Test stage completed successfully.'
      }
    }

    stage('Deploy') {
      steps {
        echo 'Deploy stage completed successfully.'
      }
    }
  }

  post {
    success {
      echo 'Pipeline completed successfully.'
    }
  }
}
```

Figure 16 — Jenkinsfile source defining Build, Test and Deploy.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git push
Username for 'https://github.com': kunalprajapati14042005
Password for 'https://kunalprajapati14042005@github.com':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 443 bytes | 147.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/kunalprajapati14042005/lab1-devops-version-control.git
 f7e9d4f..84e1e12  main -> main
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ |
```

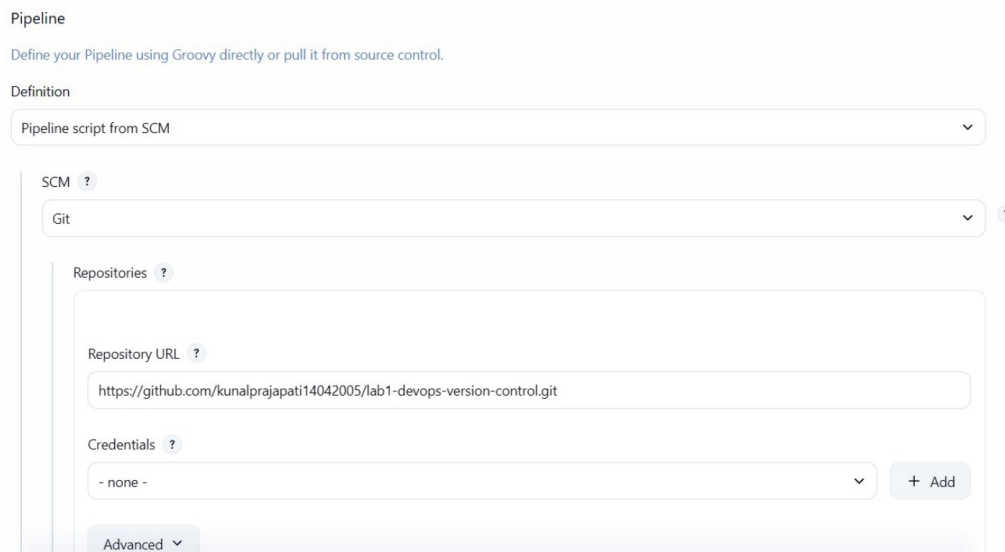
Figure 17 — Jenkinsfile commit/push evidence.

Pipeline flow

Build → Test → Deploy → Post-build success/failure handling

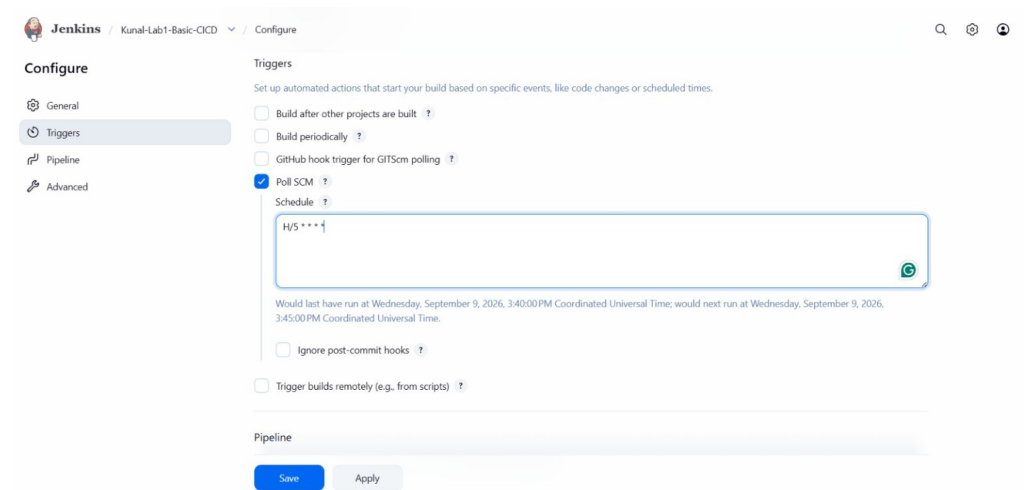
13. SCM Pipeline & Automated Trigger

Jenkins was configured to load the Jenkinsfile from Git SCM and use Poll SCM for automated repository checking.



The screenshot shows the Jenkins Pipeline configuration page. At the top, it says "Pipeline" and "Define your Pipeline using Groovy directly or pull it from source control." Below this, the "Definition" section has a dropdown menu set to "Pipeline script from SCM". Under the "SCM" section, the "Git" option is selected. The "Repositories" section contains a "Repository URL" field with the value "https://github.com/kunalprajapati14042005/lab1-devops-version-control.git". The "Credentials" section has a dropdown menu set to "- none -" and an "Add" button. At the bottom, there is an "Advanced" dropdown menu.

Figure 18 — Pipeline script from SCM configuration.



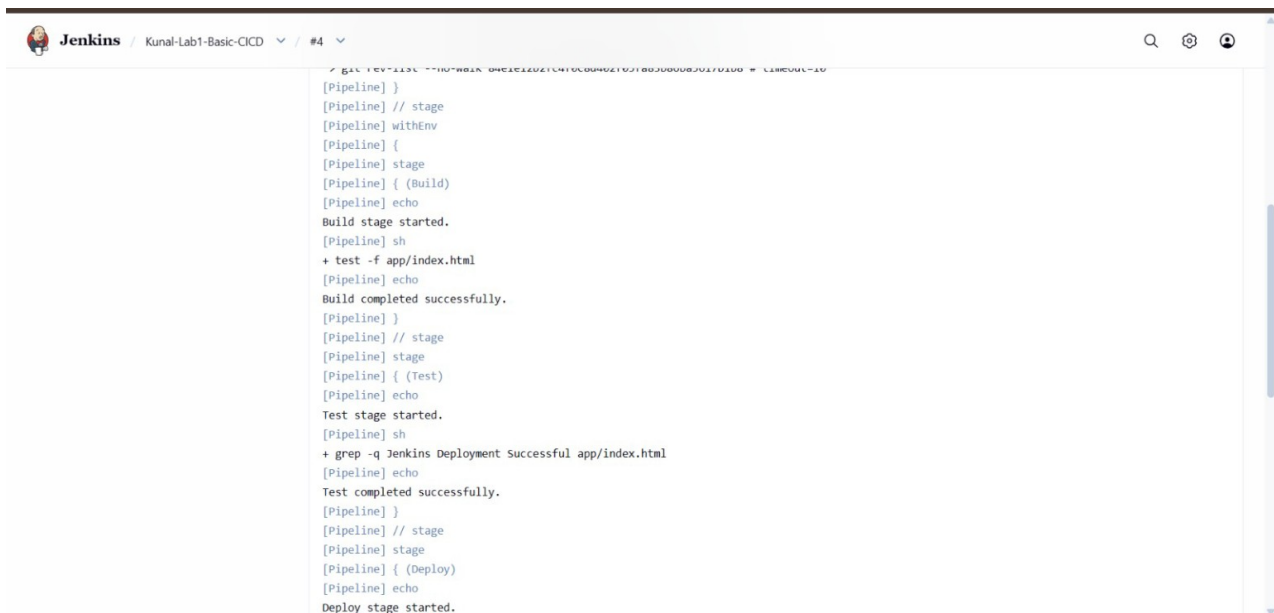
The screenshot shows the Jenkins configuration page for "Kunal-Lab1-Basic-CICD". The "Configure" section is active, and the "Triggers" tab is selected. Under "Triggers", the "Poll SCM" option is checked. The "Schedule" field is set to "H/5 * * * *". Below the schedule field, it says "Would last have run at Wednesday, September 9, 2026, 3:40:00 PM Coordinated Universal Time; would next run at Wednesday, September 9, 2026, 3:45:00 PM Coordinated Universal Time." There are also checkboxes for "Ignore post-commit hooks" and "Trigger builds remotely (e.g., from scripts)". At the bottom, there are "Save" and "Apply" buttons.

Figure 19 — Poll SCM trigger configuration.

Observation: Jenkins can obtain the pipeline definition from the main branch and poll the repository for changes.

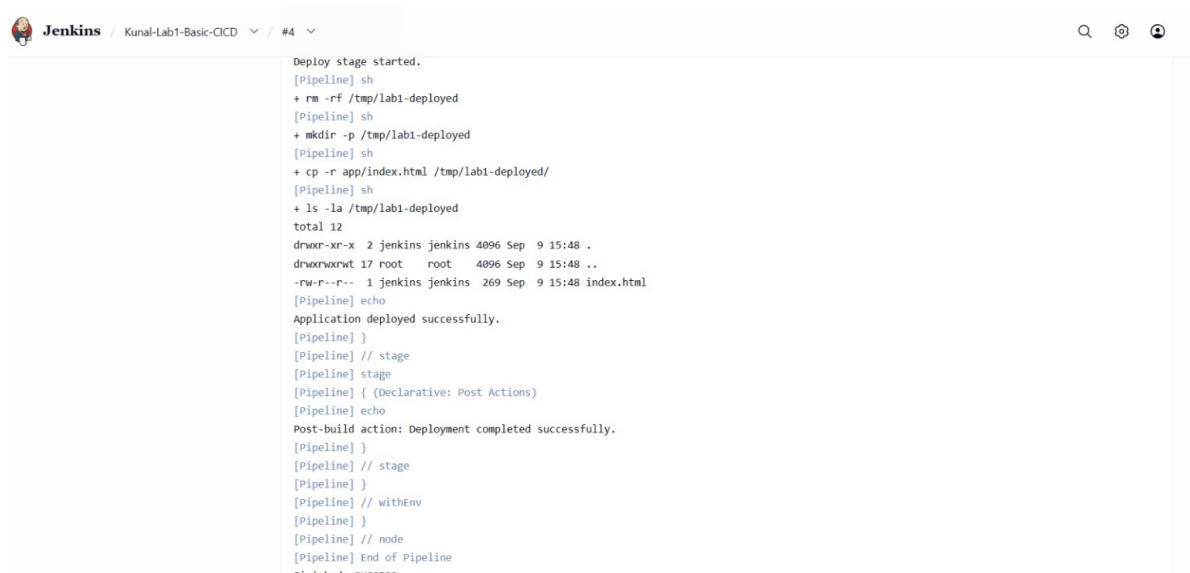
14. Build, Test & Deploy Execution

The deployment pipeline validates the application before copying it to the deployment directory.

A screenshot of the Jenkins web interface showing the console output for a pipeline run. The interface includes the Jenkins logo, the job name 'Kunal-Lab1-Basic-CICD', and a dropdown menu showing '#4'. The console output is displayed in a monospaced font, showing the execution of the Build and Test stages. The Build stage starts with 'Build stage started.', followed by a shell command 'sh' and a test command 'test -f app/index.html'. The output shows 'Build completed successfully.' The Test stage starts with 'Test stage started.', followed by a shell command 'sh' and a grep command 'grep -q Jenkins Deployment Successful app/index.html'. The output shows 'Test completed successfully.' The Deploy stage starts with 'Deploy stage started.'.

```
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Build stage started.
[Pipeline] sh
+ test -f app/index.html
[Pipeline] echo
Build completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Test stage started.
[Pipeline] sh
+ grep -q Jenkins Deployment Successful app/index.html
[Pipeline] echo
Test completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploy stage started.
```

Figure 20 — Build and Test stages completed successfully.

A screenshot of the Jenkins web interface showing the console output for the Deploy stage and post-build success action. The interface includes the Jenkins logo, the job name 'Kunal-Lab1-Basic-CICD', and a dropdown menu showing '#4'. The console output is displayed in a monospaced font, showing the execution of the Deploy stage and post-build success action. The Deploy stage starts with 'Deploy stage started.', followed by a shell command 'sh' and a series of commands: 'rm -rf /tmp/lab1-deployed', 'mkdir -p /tmp/lab1-deployed', 'cp -r app/index.html /tmp/lab1-deployed/', and 'ls -la /tmp/lab1-deployed'. The output shows the directory listing for /tmp/lab1-deployed, which includes 'total 12', 'drwxr-xr-x 2 jenkins jenkins 4096 Sep 9 15:48 .', 'drwxrwxrwt 17 root root 4096 Sep 9 15:48 ..', and '-rw-r--r-- 1 jenkins jenkins 269 Sep 9 15:48 index.html'. The output shows 'Application deployed successfully.' The post-build action is 'Deployment completed successfully.'.

```
Deploy stage started.
[Pipeline] sh
+ rm -rf /tmp/lab1-deployed
[Pipeline] sh
+ mkdir -p /tmp/lab1-deployed
[Pipeline] sh
+ cp -r app/index.html /tmp/lab1-deployed/
[Pipeline] sh
+ ls -la /tmp/lab1-deployed
total 12
drwxr-xr-x 2 jenkins jenkins 4096 Sep 9 15:48 .
drwxrwxrwt 17 root root 4096 Sep 9 15:48 ..
-rw-r--r-- 1 jenkins jenkins 269 Sep 9 15:48 index.html
[Pipeline] echo
Application deployed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Post-build action: Deployment completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 21 — Deploy stage and post-build success action.

Observation: successful Build and Test execution allowed the Deploy stage to complete.

15. Deployment & Browser Verification

The deployed application was served locally and verified through both terminal and browser evidence.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ cd /tmp/lab1-deployed
kunal_prajapati@LAPTOP-45509TK7:/tmp/lab1-deployed$ python3 -m http.server 8000 --bind 0.0.0.0 >/tmp/lab1-server.log 2>&
[1] 4025
kunal_prajapati@LAPTOP-45509TK7:/tmp/lab1-deployed$ echo $!
4025
kunal_prajapati@LAPTOP-45509TK7:/tmp/lab1-deployed$ curl http://localhost:8000
<!DOCTYPE html>
<html>
<head>
  <title>Lab 1 Jenkins Deployment</title>
</head>
<body>
  <h1>Jenkins Deployment Successful</h1>
  <p>Lab 1 - DevOps Foundations & Continuous Integration</p>
  <p>Application deployed automatically by Jenkins.</p>
</body>
</html>
kunal_prajapati@LAPTOP-45509TK7:/tmp/lab1-deployed$ |
```

Figure 22 — Local HTTP server and curl verification of the deployed application.

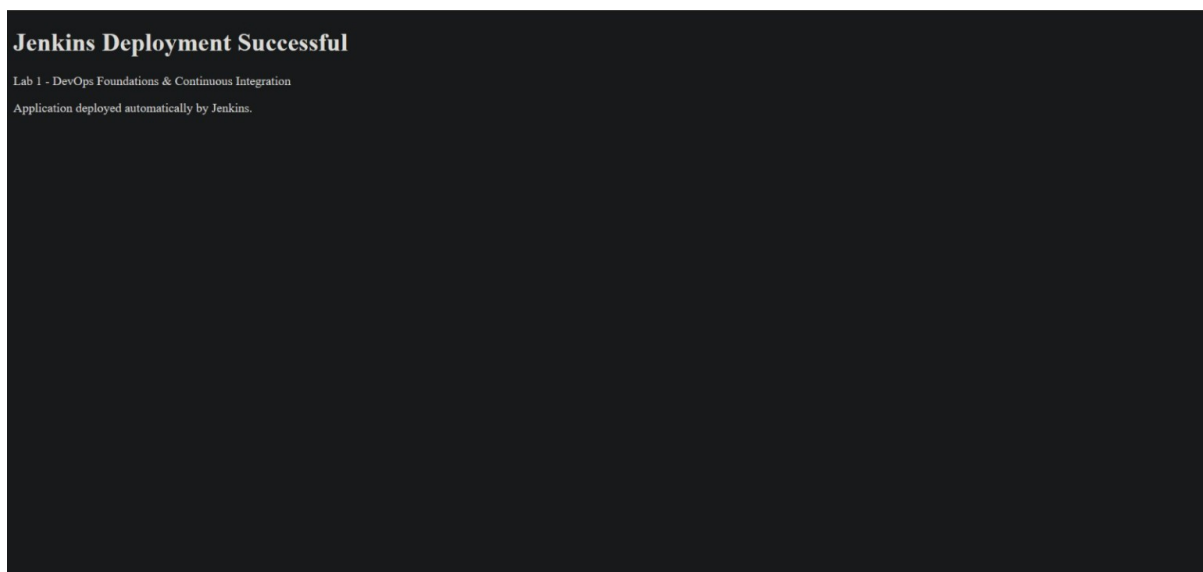


Figure 23 — Browser verification showing Jenkins Deployment Successful.

Commands

```
cd /tmp/lab1-deployed
python3 -m http.server 8000 --bind 0.0.0.0
curl http://localhost:8000
```


16. Final Pipeline Result & Stage View

The Jenkins pipeline completed successfully across its configured stages.

```
[Pipeline] { (Test)
[Pipeline] echo
Test stage completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploy stage completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 24 — Jenkins pipeline completion with SUCCESS.

Stages

September 9, 2026

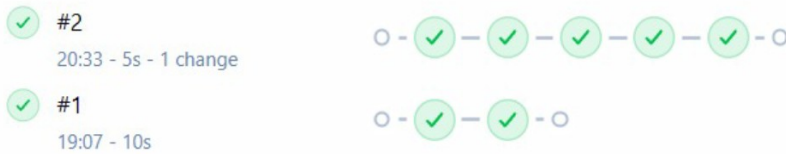


Figure 25 — Stage View showing successful pipeline stages.

Observation: the final CI/CD workflow produced a successful automated execution and verified deployment.

17. Consolidated Command Record

Core commands recorded across the Lab 1 experiments:

Environment

```
lsb_release -a
whoami
date
```

Tools

```
git --version
java -version
jenkins --version
```

Jenkins

```
sudo systemctl status jenkins --no-pager
```

Repository

```
cd ~/lab1-devops
pwd
ls -la
git status
```

History

```
git log --oneline --all --decorate --graph
```

GitHub

```
git remote -v
git push origin main
```

Deployment verification

```
cd /tmp/lab1-deployed
python3 -m http.server 8000 --bind 0.0.0.0
curl http://localhost:8000
```

Note: this report records the commands and evidence actually used for the final Lab 1 project. Flask/virtual-environment commands are not claimed because the final project evidence is the static web application under app/.

18. Observations & Task 1 Requirement Mapping

Portal requirement: Record all commands, pipeline configurations, screenshots, commit history, and observations for each experiment.

Requirement	Evidence in this report
Commands	Environment, Git, Java, Jenkins, repository, history, remote, deployment verification.
Pipeline configurations	Basic Jenkins Pipeline, Jenkinsfile from SCM, Poll SCM trigger, Build/Test/Deploy stages.
Screenshots	Terminal, Jenkins dashboard/configuration, console output, Stage View and browser verification.
Commit history	Setup, documentation, pipeline/Jenkinsfile and deployment commits shown through Git history.
Observations	Tools verified; Jenkins active; GitHub synchronized; pipeline successful; deployment verified.

Conclusion

The evidence provides a consolidated trail from Ubuntu/tool verification through Git version control, GitHub synchronization, Jenkins configuration, automated Build/Test/Deploy execution and deployment verification. This directly addresses the Task 1 requirement.